BLOCKCHAIN LABORATORY

Experiment No. 7

Introduction to Truffle, Establishing a Local Blockchain and Deployment of Smart Contract

Name: Nikhil Sunchu

Roll No.: 31

Class/section: D20A

Date: 20 October 2026

1. Aim

Introduction to Truffle, establishing a local blockchain using Ganache, and deployment of a smart contract.

2. Theory

2.1. 1. Truffle Framework

Truffle is a development framework for Ethereum-compatible smart contracts. It provides commands to initialize a project, compile Solidity contracts, manage deployment scripts called migrations, and interact with deployed contracts through a console. A Truffle project separates contract source files, migration scripts, tests, build artifacts, and network configuration.

2.2. 2. Ganache Local Blockchain

Ganache provides a local Ethereum-compatible blockchain for development and testing. It creates test accounts funded with simulated Ether and exposes a JSON-RPC endpoint through which development tools can submit transactions and query blockchain data. The local chain used in this experiment was available at `127.0.0.1:8545` with chain ID `1337`.

2.3. 3. Solidity Smart Contracts

A smart contract is a program deployed to a blockchain. Solidity is a statically typed programming language used to implement contracts on Ethereum-compatible networks. The `StudentRecord` contract stores a student's name and roll number, initializes them through a constructor, and provides functions to update the values.

2.4. 4. Compilation, Migration and Deployment

Truffle compiles Solidity source code into contract artifacts, including the ABI and bytecode. A migration script specifies how a contract is deployed and which constructor arguments are supplied. When a migration is run against a configured network, Truffle submits the deployment transaction and reports the resulting contract address, block number, and gas usage.

2.5. 5. Transactions and Contract State

Reading public state variables through getter functions does not change the blockchain state. Calling state-changing functions submits transactions that are recorded in blocks

and update the stored values. The contract state can be checked through the Truffle console after deployment and after subsequent transactions.

3. Tools and Environment Used

- Operating environment: Linux terminal
- Node.js version: 22.23.3; npm version: 10.9.9
- Bun used to manage project dependencies and execute commands
- Truffle version: 5.11.5
- Ganache version: 7.9.1
- Solidity compiler: 0.8.21
- Project configuration: `truffle-config.js`
- Contract file: `contracts/StudentRecord.sol`
- Migration file: `migrations/2_deploy_student_record.js`

4. Tasks Performed

4.1. Task 1 – Initialize the Truffle Project

- Installed Node.js and npm using the existing development environment and installed Truffle locally using Bun.
- Initialized the project using `bunx truffle init`.
- Verified that the project contained the `contracts`, `migrations`, and `test` directories, along with `truffle-config.js`.

4.2. Task 2 – Establish the Local Blockchain

- Started Ganache using `bunx ganache`.
- Confirmed that Ganache created ten test accounts and started its RPC server at `127.0.0.1:8545`.
- Used chain ID `1337` for the local development network.

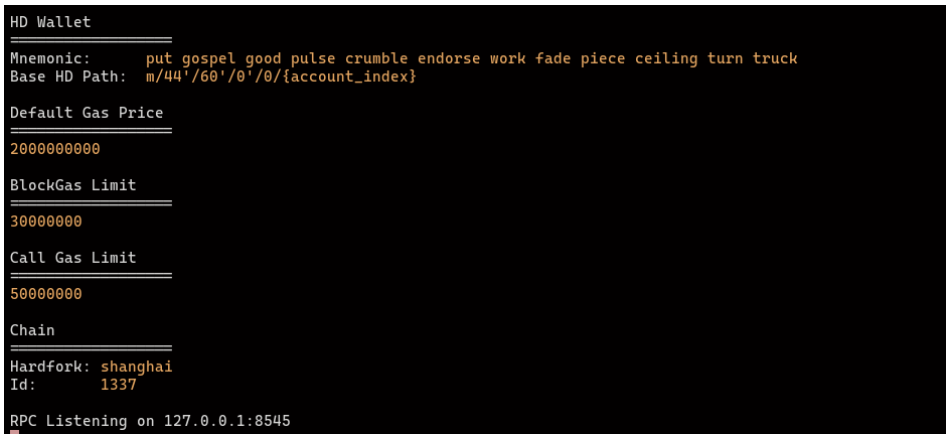


Figure 1: Ganache local blockchain running and listening on the RPC endpoint

4.3. Task 3 – Write the Solidity Smart Contract

Created `contracts/StudentRecord.sol` with two public state variables, a constructor, and two functions for updating the student's name and roll number. The contract source code is shown below.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract StudentRecord {
    string public studentName;
    uint256 public studentRoll;

    constructor(string memory _name, uint256 _roll) {
        studentName = _name;
        studentRoll = _roll;
    }

    function updateName(string memory _name) public {
        studentName = _name;
    }

    function updateRoll(uint256 _roll) public {
        studentRoll = _roll;
    }
}
```

4.4. Task 4 – Compile the Contract

- Ran `bunx truffle compile` from the project directory.
- Truffle compiled `StudentRecord.sol` using Solidity compiler 0.8.21.

- The contract artifacts were written to `build/contracts`.

```
Blockchain/Exp7 ❯ bunx truffle compile                    main
Compiling your contracts...
===========================
> Compiling ./contracts/StudentRecord.sol
> Artifacts written to /home/darkhat/Documents/sem7/Blockchain/Exp7/build/contracts
> Compiled successfully using:
   - solc: 0.8.21+commit.d9974bed.Emscripten.clang
Blockchain/Exp7 ❯
```

Figure 2: Successful compilation of StudentRecord using Truffle

4.5. Task 5 – Configure the Network and Verify the Connection

Configured the Truffle development network to use the local Ganache RPC endpoint. The relevant configuration was:

```
networks: {
  development: {
    host: "127.0.0.1",
    port: 8545,
    network_id: "*",
  },
},
```

The Truffle console was opened with `bunx truffle console -network development`. The chain ID and number of available accounts were checked using the following commands:

```
web3.eth.getChainId()
(await web3.eth.getAccounts()).length
```

The returned chain ID was `1337`, and ten accounts were available.

```
truffle(development)> web3.eth.getChainId()
1337
truffle(development)> (await web3.eth.getAccounts()).length
10
truffle(development)>
```

Figure 3: Verification of the Ganache chain ID and available accounts in Truffle

4.6. Task 6 – Create the Migration Script

Created `migrations/2_deploy_student_record.js` to deploy the contract with the initial student name and roll number. The migration source code is shown below.

```
const StudentRecord = artifacts.require("StudentRecord");

module.exports = function (deployer) {
  deployer.deploy(StudentRecord, "Nikhil Sunchu", 31);
};
```

4.7. Task 7 – Deploy the Smart Contract

- Executed `bunx truffle migrate -network development`.
- Truffle ran the migration and deployed `StudentRecord` to Ganache.
- The deployment was recorded in block 1. Truffle reported gas usage of 514262 and a total deployment cost of 0.00173563425 ETH.

```
Blockchain/Exp7 ❯ bunx truffle migrate --network development
This version of µWS is not compatible with your Node.js build:

Error: Cannot find module '../binaries/uws_linux_x64_127.node'
Require stack:
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/@trufflesuite/uws-js-unofficial/src/uws.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/ganache/dist/node/core.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/truffle/build/migrate.bundled.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/original-require/index.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/truffle/build/cli.bundled.js
Falling back to a NodeJS implementation; performance may be degraded.


Compiling your contracts...
===========================
> Everything is up to date, there is nothing to compile.


Starting migrations...
======================
> Network name:    'development'
> Network id:      1792498646161
> Block gas limit: 30000000 (0x1c9c380)


2_deploy_student_record.js
==========================

   Deploying 'StudentRecord'
   -------------------------
   > transaction hash:    0xe3ff7122874de7688635c79b0a5ad2e9fcc87b4b9b5ffa8c0ac430df9089b73c
   > Blocks: 0            Seconds: 0
   > contract address:    0xE3464dFA0Cc235246965be2394f9020E215a81c2
   > block number:        1
   > block timestamp:     1792499025
   > account:             0x940215Db42921c69e63276E70107B5Cd7e9b7C14
   > balance:             999.99826436575
   > gas used:            514262 (0x7d8d6)
   > gas price:           3.375 gwei
   > value sent:          0 ETH
   > total cost:          0.00173563425 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:       0.00173563425 ETH

Summary
=======
> Total deployments:   1
> Final cost:          0.00173563425 ETH
```

Figure 4: Successful deployment of StudentRecord to the Ganache development network

The deployed contract address was `0xE3464dFA0Cc235246965be2394F9020E215a81c2`.

4.8. Task 8 – Verify the Initial Contract State

Loaded the deployed contract instance in the Truffle console and called the public getter functions.

```
const instance = await StudentRecord.deployed();
await instance.studentName();
(await instance.studentRoll()).toString();
```

The initial values returned were `Nikhil Sunchu` and `31`.

```
Blockchain/Exp7 ❯ bunx truffle migrate --network development
This version of µWS is not compatible with your Node.js build:
Blockchain/Exp7 ❯ bunx truffle console --network development
This version of µWS is not compatible with your Node.js build:

Error: Cannot find module '../binaries/uws_linux_x64_127.node'
Require stack:
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/@trufflesuite/uws-js-unofficial/src/uws.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/ganache/dist/node/core.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/truffle/build/console.bundled.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/original-require/index.js
- /home/darkhat/Documents/sem7/Blockchain/Exp7/node_modules/truffle/build/cli.bundled.js
Falling back to a NodeJS implementation; performance may be degraded.


truffle(development)> const instance = await StudentRecord.deployed()
undefined
truffle(development)> await instance.studentName()
'Nikhil Sunchu'
truffle(development)> (await instance.studentRoll()).toString()
'31'
truffle(development)>
```

Figure 5: Verification of the initial student name and roll number

4.9. Task 9 – Execute Transactions and Verify the Updated State

Submitted transactions to update the student's name and roll number, then queried the contract again.

```
await instance.updateName("Nikhil Sunchu Updated");
await instance.updateRoll(32);
await instance.studentName();
(await instance.studentRoll()).toString();
```

The transactions succeeded. The name was updated to `Nikhil Sunchu Updated` and the roll number to `32`. The name update was recorded in block 2 and the roll-number update in block 3.



Figure 6: Successful update transactions and verification of the updated contract state

The final block number and deployed contract address were checked using:

```
await web3.eth.getBlockNumber();
await instance.address;
```

The final block number was `3`, and the contract address matched the deployment output.

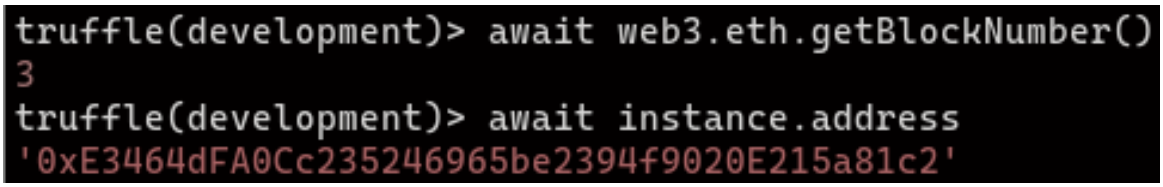


Figure 7: Final verification of the blockchain block number and contract address

5. Result

The Truffle project was initialized successfully, and a local blockchain was established using Ganache. The `StudentRecord` smart contract was compiled using Solidity 0.8.21 and deployed to the development network. The initial student name and roll number were verified as `Nikhil Sunchu` and `31`. Subsequent transactions updated the name to `Nikhil Sunchu Updated` and the roll number to `32`. The final local blockchain height was block 3, and the deployed contract address was verified.